Architecture - Ghost Data Still Exists

One-line pitch

A deletion-audit agent where **the model describes and deterministic code decides**: every verdict in `output.csv` is derived from grounded observations (resource, time, state, identifiers, evidence id) by a small, testable rule engine. Claude is used only where code cannot read the input: multilingual request intent and console screenshots. Everything else - identity, evidence, time, policy, ordering - is deterministic and explainable (`python code/main.py --cases case_11 --explain`).

Pipeline

```
deletion_requests.csv ──► intent (guard + Claude Haiku 5.5, cached) ──►
NOT_APPLICABLE / []
        │
        ▼
identity.py        email → user_id + every identifier the person ever had
                   (user_identities + user_audit_log; sha256 of every email)
        │
        ▼
evidence/*.py      94 files → Observation(resource_id, observed_at, presence,
kind,
                   pii/enabled/restore_allowed/retention, identifiers,
evidence_id)
                   logs (8 dialects, trace joins) · JSON (bundles, exports,
ledgers)
                   SQL (sqlite3 in-memory; replicas, snapshot_scope) · TXT · PDF
                   PNG (Claude Sonnet 5.5 transcription → fixed JSON schema,
cached)
        │
        ▼
linkage.py         which resources belong to this subject for this case
                   identifiers ▸ catalog LINK rows ▸ document aliases ▸
                   deletion-worker run (accounts) ▸ bundle order (anonymised
analytics)
                   + temporal relevance window (one person can have several cases)
        │
        ▼
state.py           state at audit_at = latest observation the auditor could have
seen
                   (file capture time AND record time ≤ audit_at); replica copies
count;
                   scoped snapshots imply absence only for the people they mention
        │
        ▼
policy.py          deletion_rules.csv: kind → category → DELETE / REDACT /
ANONYMIZE /
                   DISABLE / BLOCK_RESTORE / RETAIN; compare with state → steps
        │
```

```
        ▼
planner.py          COMPLETE (verification with proof) | INCOMPLETE (findings +
                    remediation + verification); order = phase ▸ operation table ▸
                    system_id ▸ resource_id; consecutive step numbers; JSON
        |
        ▼
agent.py            reviewer loop (Claude Sonnet 5.5) on uncertain cases: tools
                    search_evidence / read_source / lookup_activity /
resolve_identity,
                    iteration cap, strict submit_review schema; vetoes and
proposals are
                    accepted only when code re-reads the cited source and confirms
it;
                    accepted changes re-enter planner.py (never written directly)
```

Where the model acts, and where it cannot

stage	model	what it may do	what code checks
intent	Haiku 5.5	label DELETION / NON_DELETION for untrusted request text	closed enum; deterministic guard flags disagreement
screenshots	Sonnet 5.5	transcribe console UI into typed records	schema + vocabulary; records without a second source are ignored
review	Sonnet 5.5 + tools	investigate a draft, veto a finding, propose a missed resource	cited source must exist, predate audit_at, name the resource, contain the verbatim quote and a compliance marker

The reviewer is a real agent loop (model-driven tool selection, up to 8 iterations, strict tool schemas, typed retries with exponential backoff on top of the SDK retries, timeouts, cached transcripts, every final answer validated against `schemas.REVIEW_SCHEMA`), but it has no write access: the deterministic planner recomputes the case with the validated inputs.

Key design decisions

Decision	Why
Observations as the single intermediate representation	20+ input formats collapse into one model; every downstream rule is format-agnostic and unit-testable.
Reported statuses / activity results are never proof	The problem says they are claims. `system_activity.csv` is used only to *discover* candidate resources and as a last-resort citation.
Time is enforced twice	A record is usable only if its own timestamp **and** its file's `captured_at` precede `audit_at` (bundles contain older snapshots inside newer files).

Decision	Why
Late evidence may link, never judge	Ownership of a resource is a static fact; the state and citations come only from evidence the auditor could see at `audit_at` (`config.USE_LATE_EVIDENCE_FOR_LINKAGE`).
Replica drift = still present	Any copy in any replica table of a snapshot counts (`customer_accounts_failover_replica`: primary row gone, failover row alive).
Scoped absence only for the people a snapshot mentions	Inventory / replica captures list the records of the users under review; a missing row proves nothing for other users.
Kind beats system	`analytics_9914` keyed only by the hash of an old email is a SECONDARY_IDENTIFIER (delete), not an analytics profile (anonymise); `support_*` can be a message (redact), an attachment or a legacy contact entry (delete).
Screenshot-only records need a second source	The data catalog (`LINK_RESOURCE_IDENTITY`) ties each intended screenshot record to a person; records seen only in one screenshot without any other mention are ignored (console noise).
Deterministic ordering and serialisation	`step_sort_key` implements the contract literally; the validator re-checks every row before submission.
Agent as reviewer, not as decider	Uncertain cases (low-confidence links, screenshot states, unproven resources) get a tool-using second opinion; its claims are re-verified against the raw evidence so a hallucinated citation cannot change `output.csv`.

Security and robustness

- `request_text` and screenshot contents are untrusted. They are classified or transcribed, never followed; the system prompts say so explicitly and model outputs are validated against closed vocabularies before use.
- The intent and screenshot calls expose no tools. The reviewer's four tools are read-only and return JSON; it cannot read arbitrary files, run code or change state. Code decides, so a prompt injection can at most flip one classification or file a citation that code rejects; the deterministic guard flags any intent disagreement for review (`intent.note`).
- Every model boundary is typed: JSON schemas in `schemas.py` (intent, vision, review) plus closed vocabularies. A non-conforming answer is an error, never a verdict.
- Evidence files are untrusted input to the parsers as well, not only to the prompts. SQL snapshots run in an in-memory SQLite connection under an authorizer that denies ATTACH / DETACH / extension loading and every PRAGMA except `foreign_keys`; a script that contains a file-access statement (ATTACH, VACUUM INTO) is rejected, logged and skipped, so a crafted snapshot cannot read or write files on the auditor's machine. Reviewer tool pointers are resolved only through the evidence manifest: no string from the model ever selects a file path.
- Retries are typed and bounded: `llm.with_retries` retries rate limits, overloads, 5xx and transport errors with exponential backoff and jitter (4 attempts, 30 s cap) on top of the SDK's own retries; 4xx errors are never retried; refusals are detected through `stop_reason` and recorded, not swallowed.

- Structured logging (`log.py`): one `key=value` record per model call, retry, reviewer verdict and progress tick; `--log-level` on the CLI.
- Secrets only from the environment (`.env`, git-ignored). No PII leaves the pipeline except the request text sent for classification; evidence text is never sent to the model (only screenshots, which contain fictional data by design of the challenge).
- Every model call is cached on disk keyed by model + prompt + content and logged in `code/evaluation/usage_ledger.jsonl`, so the full run is reproducible and auditable.
- Degrades gracefully: without an API key the engine still runs (`--no-llm`) using the deterministic intent guard and any cached extractions.

Code quality

- `pyproject.toml` configures ruff (pyflakes, pycodestyle, isort, pyupgrade, bugbear, comprehensions, simplify, return, pylint complexity: at most 14 branches, 60 statements and 8 returns per function) and pytest; `ruff check` and `ruff format --check` are clean.
- Full type hints on public functions and dataclasses for every record that crosses a module boundary (`Observation`, `Scope`, `Link`, `ResourceState`, `ResourcePlan`, `Step`, `CaseResult`, `Review`).
- Tests: 16 unit tests (log dialects, identity, state, policy, ordering, reviewer tools, code-side validation of vetoes and proposals), 6 custom evaluation cases, the sample regression (status accuracy 1.0, step F1 0.971) and the contract validator on `output.csv`.
- Module map and guardrails checklist: `code/README.md`; prompt design: `code/audit/prompts/README.md`.

Known limitations

- Some canonical steps are not derivable from participant data (e.g. a retained security record that no file mentions); we never invent resource ids.
- Anonymised analytics profiles carry no identifier; we attribute them only when the bundle order + deletion-worker run leave exactly one consistent assignment.
- When several cases share a deletion-worker run and none of the evidence names their deleted accounts, account attribution falls back to case order (documented heuristic).
- `resource_id` ordering within a bucket is lexicographic (the sample never has two resources of the same system and operation, so the contract is ambiguous there).